EXP 5: Analyze the impact of Zero-shot, Few-shot, and Chain-of-Thought (COT) prompting on the reasoning capabilities of Large Language Models (LLMs) and identify patterns in logical fallacies or "hallucinations"

Dataset: SQUAD dataset or GSM8K (Math Word Problems) dataset.

PROGRAM:

```
# ========================================
# GEN AI EXP 5 – FAST VERSION
# Zero-shot vs Few-shot vs COT
# GSM8K + TinyLlama
# ========================================

!pip install -q transformers torch accelerate pandas pyarrow datasets

# IMPORTS
import torch
import pandas as pd
import re
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM

# ========================================
# STEP 1 – DOWNLOAD DATASET
# ========================================
print("Downloading GSM8K...")
ds = load_dataset("gsm8k","main")
data = ds["train"].to_pandas()
print("Dataset size:", len(data))

# ========================================
# STEP 2 – DEVICE
# ========================================
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using:", device)

# ========================================
# STEP 3 – LOAD MODEL
# ========================================
model_name="TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer=AutoTokenizer.from_pretrained(model_name)

model=AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if device=="cuda" else torch.float32
).to(device)

model.eval()
```

```python
# ========================================
# STEP 4 — PROMPT BUILDER
# ========================================
def build_prompt(question,mode="zero_shot",examples=None):

    if mode=="zero_shot":
        return f"Solve the math problem. Give only number.\n\nQuestion: {question}\nAnswer:"

    elif mode=="few_shot":
        p="Examples:\n\n"
        for q,a in examples:
            p+=f"Q: {q}\nA: {a}\n\n"
        p+=f"Q: {question}\nA:"
        return p

    elif mode=="cot":
        return f"Solve step by step.\nQuestion: {question}\nLet's think step by step."

# ========================================
# STEP 5 — GENERATION (FAST)
# ========================================
def generate_answer(prompt):

    inputs=tokenizer(prompt,return_tensors="pt").to(device)

    with torch.no_grad():
        out=model.generate(
            **inputs,
            max_new_tokens=64,   # reduced for speed
            do_sample=True,
            temperature=0.7
        )

    return tokenizer.decode(out[0],skip_special_tokens=True)

# ========================================
# STEP 6 — NUMBER EXTRACTION
# ========================================
def extract_last_number(text):
    nums=re.findall(r"-?\d+\.?\d*",text)
    return nums[-1] if nums else None
```

```python
# ========================================
def get_examples(k=3):
    ex=[]
    for i in range(k):
        q=data.iloc[i]["question"]
        a=data.iloc[i]["answer"].split("####")[-1].strip()
        ex.append((q,a))
    return ex

few_examples=get_examples(3)

# ========================================
# STEP 8 — EVALUATION
# ========================================
def evaluate(mode,n=10):

    correct=0
    hall=0

    for i in range(n):

        q=data.iloc[i]["question"]
        gt=data.iloc[i]["answer"].split("####")[-1].strip()

        if mode=="few_shot":
            prompt=build_prompt(q,mode,few_examples)
        else:
            prompt=build_prompt(q,mode)

        out=generate_answer(prompt)

        pred=extract_last_number(out)
        gt=extract_last_number(gt)

        if pred is None:
            hall+=1
        elif pred==gt:
            correct+=1

        print(mode,"processed",i+1,"/",n)

    acc=correct/n
    hall_rate=hall/n

    print("\nMode:",mode)
    print("Accuracy:",acc)
    print("Hallucination:",hall_rate)

    return acc,hall_rate

#
```

```
# ========================================
# STEP 9 — RUN EXPERIMENT (FAST)
# ========================================
acc_zero,h_zero=evaluate("zero_shot",10)
acc_few,h_few=evaluate("few_shot",10)
acc_cot,h_cot=evaluate("cot",10)
```

OUTPUT:

```
Downloading GSM8K...
Dataset size: 7473
Using: cpu
Loading weights: 100% ████████████████████ 201/201 [00:00<00:00, 276.05it/s, Materializing param=model.norm.weight]
zero_shot processed 1 / 10
zero_shot processed 2 / 10
zero_shot processed 3 / 10
zero_shot processed 4 / 10
zero_shot processed 5 / 10
zero_shot processed 6 / 10
zero_shot processed 7 / 10
zero_shot processed 8 / 10
zero_shot processed 9 / 10
zero_shot processed 10 / 10

Mode: zero_shot
Accuracy: 0.0
Hallucination: 0.0
few_shot processed 1 / 10
few_shot processed 2 / 10
few_shot processed 3 / 10
few_shot processed 4 / 10
few_shot processed 5 / 10
few_shot processed 6 / 10
few_shot processed 7 / 10
few_shot processed 8 / 10
few_shot processed 9 / 10
few_shot processed 10 / 10

Mode: few_shot
Accuracy: 0.0
Hallucination: 0.0
cot processed 1 / 10
cot processed 2 / 10
cot processed 3 / 10
cot processed 4 / 10
cot processed 5 / 10
cot processed 6 / 10
cot processed 7 / 10
cot processed 8 / 10
cot processed 9 / 10
cot processed 10 / 10

Mode: cot
Accuracy: 0.0
Hallucination: 0.0
```